# SOP-OPS-001 · How a change travels: preview, Promote, main, the public twin

SOP OPS-001 | Status draft | Changed 2026-11-01 | For Everyone

**Department:** OPS · **ID:** 001 · **Status:** draft (approved when tagged) · **Owner:** repository steward · **Last changed:** 2026-11-01

## Purpose

So that anyone reading a page knows which copy they are looking at, and so that nothing reaches the public unless someone put its path on a list.

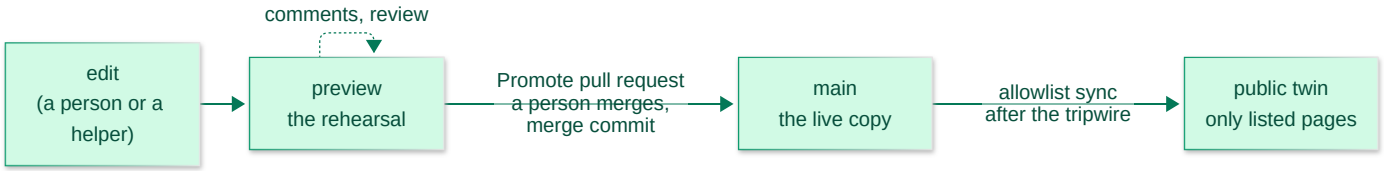
## Scope

Every repository in the fleet that has a `preview` branch (sites, drafting desks, Bushido). Organisation `proposals` repositories without a `preview` branch use pull requests into `main` directly; the public-twin rule still applies.

## Roles

| Role | Does |
|---|---|
| Author (person or helper) | edits on `preview`, opens the Promote pull request |
| Reviewer | reads on `preview`, comments |
| Merger (a human with the role) | merges the Promote; never a helper |
| Steward | keeps the allowlist and the deploy key |

## The road




edit · `preview` · Promote · `main` · public twin (SOP-OPS-003 has the twin's rule)

## Two things called "preview"

The fleet uses the word *preview* for two different things. They often sit side by side in one repository, which is why they get confused. If you have never used a `preview` branch before, start here.

### From zero: what a branch is

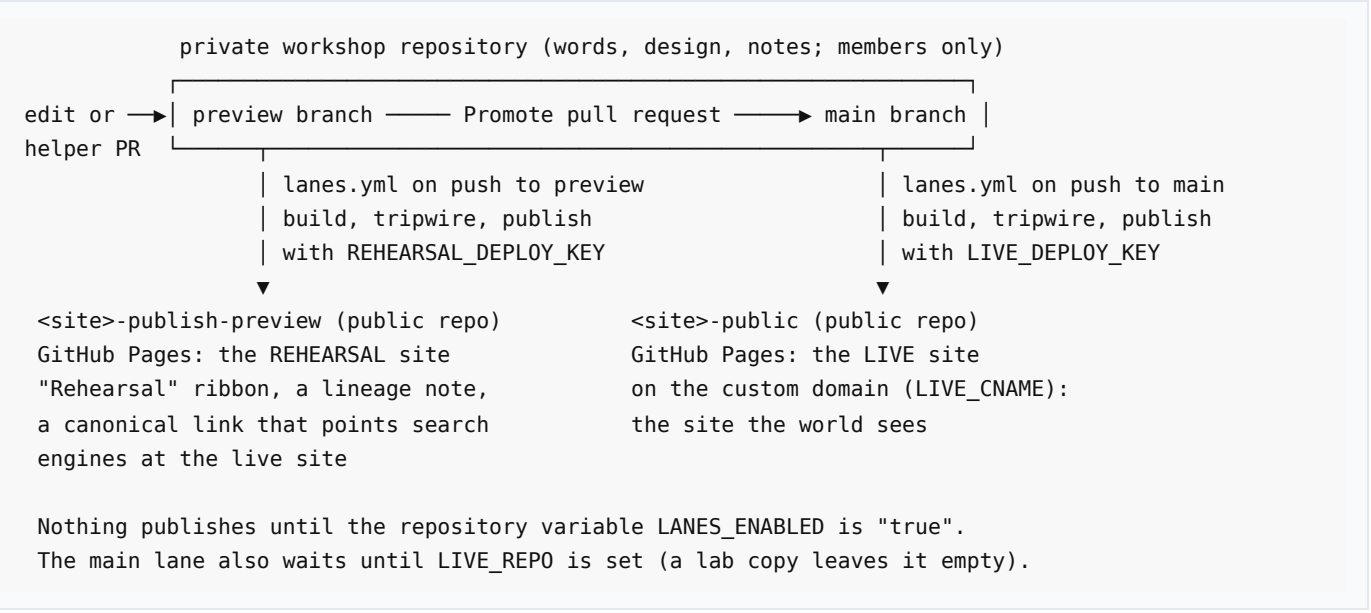
A repository is a folder with its whole history. A **branch** is one named line of that history. Every repository has `main`, the copy everyone treats as real. A repository may also keep a second line called `preview`: the same files, plus whatever changes have not been accepted yet. Switching branches on GitHub (the branch picker above the file list, which says `main` until you change it) shows you the other copy. Nothing is lost by switching; you are only choosing which copy to look at.

### Thing one: the `preview` branch (the repository's rehearsal)

- **What it is:** a branch inside one repository where changes land first. Authors and helpers commit there; reviewers read there.
- **How it moves on:** a pull request from `preview` into `main`, titled `Promote: <what>` (SOP-OPS-002). A human with the role merges it with a merge commit; then `main` carries the change.
- **What it publishes:** nothing by itself. It is a holding place for words, code and procedures.
- **Where you find it:** in proposals desks, SOP repositories, Bushido, and every site workshop. On GitHub: the branch picker, or `https://github.com/<owner>/<repo>/tree/preview`.
- **The tell:** a link to `main` shows the old text while your helper says the change is done. It is on `preview`; nobody has merged the Promote yet.

### Thing two: the publish-preview *lanes* (a website's rehearsal site)

A **site** repository goes one step further: a workflow ( `.github/workflows/lanes.yml` ) builds the website from **each** branch and publishes it to a **separate public repository** that serves it with GitHub Pages. Two branches, two websites:




- **Each public repository has its own write deploy key.** The private half lives in the workshop as a secret ( `REHEARSAL_DEPLOY_KEY`, `LIVE_DEPLOY_KEY` ); the public half is a deploy key on that one public repository. A deploy key opens exactly one repository and never expires, so a key that leaks cannot reach anything else, and publishing cannot stop silently the way an expired personal token does.
- **The tripwire** runs inside the build: research folders, env files or key material in the output fail the build before anything is published.
- **The switch is `LANES_ENABLED`.** Until it is `true`, the workflow file can sit in a repository and do nothing. That is how a lab copy of a site carries the file safely.
- **Every other setting is a repository variable,** not code: `LIVE_REPO` and `REHEARSAL_REPO` (where each lane publishes), `LIVE_SITE_URL`, `LIVE_BASE_PATH`, `LIVE_CNAME`, `REHEARSAL_SITE_URL`, `REHEARSAL_BASE_PATH`, `REHEARSAL_LINEAGE`, and `NEXT_PUBLIC_FORMSPREE_ENDPOINT`. A site's `transition-package/keys-and-locks.md` lists each one, what it does, who holds it and how to replace it, and never a value of a secret.

### The admin panel and the form, on a site with lanes

- **The admin panel ("the cockpit")** is part of the site itself: the key icon at the foot of every page, or Ctrl+Shift+X. It shows what is on Live and on Rehearsal, offers ready-made prompts to paste into an AI helper ("on the preview branch, change …, then open a pull request into preview"), an **Edit the words** button that opens the right file on GitHub **on the `preview` branch**, and **Promote rehearsal to live**, which opens a ready-made pull request from `preview` into `main`. Its passcode is a courtesy gate on a static site, not access control: what a person can actually change is decided by their GitHub permissions. The panel manages content (FAQ and page text), not applications.

- **Applications go to Formspree, not to the site.** The form posts to a public Formspree endpoint (the `NEXT_PUBLIC_FORMSPREE_ENDPOINT` variable; public by design, not a secret). Formspree shows each submission in its dashboard, emails it to the team inbox, and can forward it (a spreadsheet, Notion, an automation). The site stores nothing; if Formspree cannot be reached, the form falls back to the visitor's own email. Submissions are read in the Formspree dashboard, not in the admin panel.

### Side by side

| | `preview` branch | publish-preview lanes |
|---|---|---|
| What it is | a second line of history inside one repository | a workflow that builds a website from each branch |
| Exists in | desks, SOP repositories, Bushido, site workshops | site workshops only |
| What a reader sees | files on GitHub, branch picker set to `preview` | a real website (the rehearsal site, with a ribbon) |
| Moves on by | a Promote pull request, merged by a human | the same Promote: merging into `main` rebuilds the live site |
| Publishes to | nowhere | `<site>-publish-preview` (rehearsal) and `<site>-public` (live) |
| Credentials | none beyond GitHub membership | one write deploy key per public repository, held as a workshop secret |
| Switched on by | creating the branch | `LANES_ENABLED = true` (and `LIVE_REPO` set, for the live lane) |
| Search engines | not applicable | the rehearsal site points them at the live site (canonical link) |

**Worked example:** The Holy Earth Foundation's I Am One Self retreats site. The rehearsal is `theholyearthfoundation/iamoneself-retreats-publish-preview` (served at `theholyearthfoundation.github.io/iamoneself-retreats-publish-preview/`), the live site is `theholyearthfoundation/iamoneself-retreats-public` (served at `retreats.iamoneself.com`), both public; the workshop that feeds them is private to the foundation's members.

**Which one am I looking at?** A URL on `github.com` with `/tree/preview` or `/blob/preview` is the branch. A website with a "Rehearsal" ribbon is the rehearsal lane. A custom domain without a ribbon is live.

## Steps

1. Make the change on `preview`. *You should see:* the commit on the `preview` branch on GitHub; nothing on `main` yet.
2. Open a pull request from `preview` to `main` titled `Promote: <what>` (SOP-OPS-002 says how, and how a helper does it for you). *You should see:* a pull request link.
3. A reviewer reads the pages on `preview` and comments. *You should see:* comments on the pull request, or a plain yes.
4. A human with the role merges with a **merge commit**, never squash (SOP-OPS-002 explains why). *You should see:* `main` carries every commit that was on `preview`, with each author's attribution intact.
5. If the repository has a public twin, the sync copies **only the paths on the allowlist** from `main` to the twin, after a tripwire check. *You should see:* the twin updated minutes later; nothing off the list.

## Verification

`git log origin/main..origin/preview` is empty after the merge; the public twin's tree contains exactly the allowlisted paths.

## Troubleshooting

| Symptom | Likely cause | What to do |
|---|---|---|

| | | |
|---|---|---|
| The GitHub `main` link shows an old file while the local copy is new | the change is on `preview`; no Promote has been merged | open or merge the Promote (SOP-OPS-002) |
| A page is on `main` but not on the public twin | its path is not on the allowlist | add the path by pull request, on an officer's yes |
| The sync fails on the twin | the strict branch ruleset was applied to a twin that the sync force-pushes to | use the deletion-only ruleset for twins (`my-template/branch-protection/`) |

## Related

- Proposal: THEF 013 *Procedures beside proposals* (why the fleet writes SOPs and how they are approved)
- Proposal: THEF 012 *Your reading room, and how your helper joins you*
- `bushido-code/DEPLOY.md` (the topology this procedure describes), `my-template/branch-protection/README.md`, `my-template/workflow-templates/sync-public-allowlist.yml`
- A site's `.github/workflows/lanes.yml`, `transition-package/keys-and-locks.md` and `how-it-works.md` (the lanes, their keys, the form backend)
- SOP-OPS-002 (the legend of what is protected and what runs on pull requests), SOP-IT-002

## Change log

| Date | Change | By |
|---|---|---|
| 2026-10-30 | Drafted from the Oct 29 topology and the Oct 30 delivery pass | Alfred |
| 2026-10-31 | "Two things called preview": the `preview` branch versus the publish-preview lanes (`lanes.yml`, deploy keys, `LANES_ENABLED`), the admin panel and Formspree, a diagram and a side-by-side table | Alfred |
| 2026-11-01 | Lucid pass: Lucide icons on every section, the road drawn in Mermaid (on GitHub and in the PDF), SOP-OPS-003 linked for the twin's rule | Alfred |
| 2026-11-01 | ORG's copy brought level with my-template: Lucide icons in emerald, Mermaid where a picture explains, ORG's own facts kept | Alfred |

*Part of the fleet's procedures. The master copies and the legend live in my-template.*